

Ministry of Education and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory Work #10

**Automatic Control in Embedded Systems. Part 2: PID
Cooling with Comparative ON-OFF Heating**

Author:

Alexandru Cebotari

Group FAF-233

Verified by:

Alexei Martiniuc

Technical task

Purpose

This report documents the implementation of Lab 10, focused on Part 2 of the automatic-control assignment: PID control of a measurable physical parameter. The final firmware also preserves the Part 1 ON-OFF heating mode and exposes a hybrid mode in order to compare binary and continuous control strategies on the same hardware platform.

Objectives

- implement a modular PID cooling controller for temperature using a thermistor-based acquisition chain;
- preserve the ON-OFF hysteresis heating controller from Part 1 and integrate both strategies into one firmware;
- support runtime configuration of setpoint, hysteresis, acceptable range, control mode, and PID gains;
- display measured temperature, control mode, heater state, and fan duty on LCD and serial STDIO;
- send plotter-compatible `SetPoint`, `Value`, and `Output` fields for real-time observation;
- validate the combined control application through real-hardware observations.

Individual task and interpretation

The implemented hardware uses Arduino Uno, a 10k NTC thermistor divider for temperature feedback, a heater resistor and indicator LED for Part 1, and a DC motor fan controlled through an L293D driver for Part 2. The application supports three runtime modes: P1 (ON-OFF heating), P2 (PID cooling), and HYBRID (heating below setpoint and PID cooling above setpoint). Sequential periodic tasks are used instead of FreeRTOS, which is explicitly allowed by the laboratory condition.

1 1. Domain analysis

1.1 1.1 Motivation

Automatic control in embedded systems aims to keep a measurable process variable close to a desired reference. Lab 9 introduced ON-OFF control with hysteresis, a robust but coarse method. Lab 10 extends the same problem by implementing PID control, which is

more suitable for fine regulation because it can continuously adapt the actuation intensity according to the magnitude and dynamics of the error.

In thermal systems, the distinction is intuitive. A binary heater is easy to drive with ON-OFF control, but a cooling fan becomes more effective when the delivered actuation is proportional to the thermal error. Combining both approaches in one firmware highlights the practical differences between coarse switching and continuous feedback control while preserving the same sensing and reporting infrastructure.

1.2 1.2 Hardware and software context

- Hardware platform: Arduino Uno.
- Sensor subsystem: 10k NTC thermistor, 10k fixed resistor, analog acquisition on A0.
- Setpoint subsystem: potentiometer on A1 and serial command interface.
- Part 1 actuator: heating resistor and indicator LED on D8.
- Part 2 actuator: DC motor fan controlled through L293D, with D2 and D10 as command pins.
- Display subsystem: LCD1602 in 4-bit mode.
- Development resources: PlatformIO, Arduino framework, LaTeX, PlantUML.
- Monitoring interfaces: serial monitor and Arduino Serial Plotter.

The final implementation is a practical custom variant derived from the condition. The thermistor divider provides temperature feedback, the heating resistor represents binary actuation for Part 1, and the DC motor fan through L293D provides PWM-based cooling for Part 2.

1.3 1.3 Existing solutions and references

Simple thermal-control examples often stop at binary switching, while more advanced examples implement PWM fan control or heater power regulation using PID. Educational PID implementations on Arduino generally use the following structure: periodic acquisition, error computation, proportional-integral-derivative update, clamped output generation, and actuator application through PWM. The current implementation reuses these principles but integrates them with a comparative ON-OFF mode in the same application.

1.4 1.4 Problem definition

The problem is to build a modular temperature-control application that can demonstrate both binary hysteresis-based heating and PID-based cooling on the same board. The application must acquire temperature from a thermistor divider, compare it to a configurable setpoint, support mode selection, compute the corresponding control action, and report all relevant values through LCD, STDIO, and Arduino Serial Plotter.

Section conclusion

The domain analysis motivates the transition from ON-OFF control toward PID control and defines the combined Lab 10 application as a comparative, mode-selectable temperature regulation system.

2 2. Conceptualization and design

2.1 2.1 Design motivation

The design stage is required to formalize how two different control strategies can coexist in one embedded application. It defines technical requirements, clarifies the mode-selection logic, describes the hardware-software interfaces, and establishes the validation approach used to compare Part 1 and Part 2 behavior.

2.2 2.2 Technical requirements

ID	Requirement	Type	Acceptance target
R1	The application shall acquire the thermistor divider value periodically and convert it to temperature.	F	measured value is refreshed every cycle
R2	The application shall support configurable setpoint selection from potentiometer and serial commands.	F	setpoint changes during runtime
R3	The application shall provide a Part 1 ON-OFF heating mode with configurable hysteresis.	F	heater switches according to the hysteresis band
R4	The application shall provide a Part 2 PID cooling mode using a DC motor fan.	F	fan duty changes according to thermal error
R5	The application shall provide a hybrid mode that combines ON-OFF heating and PID cooling.	F	mode transitions produce coherent behavior
R6	The application shall allow runtime configuration of PID gains K_p , K_i , and K_d .	F	gains can be changed during execution
R7	The application shall allow runtime configuration of acceptable minimum and maximum temperature limits.	F	min/max thresholds are editable
R8	The application shall display measured value, mode, heater state, and fan duty on LCD and serial <code>STDIO</code> .	F	reporting remains readable and updated
R9	The application shall publish <code>SetPoint</code> , <code>Value</code> , and a signed <code>Output</code> for Arduino Serial Plotter.	F	plotter-compatible line is generated
R10	The software shall remain modular, with separated tasks, services, and drivers.	NF	source tree follows layered decomposition
R11	The execution model shall use sequential periodic tasks with justified recurrences.	NF	scheduling logic is explicit
R12	The project shall build in PlatformIO for Arduino Uno.	NF	successful build is obtained
R13	The application shall exclude functionality unrelated to automatic control of the selected process variable.	NF	implementation remains within the laboratory scope

Table 1: Technical requirements for Lab 10

2.3 2.3 System components and roles

Component	Role in the application
NTC thermistor and fixed resistor	Provide the analog signal used for temperature estimation.
Setpoint potentiometer	Provides local manual adjustment of the temperature reference.
Serial command interface	Allows mode selection and runtime configuration of setpoint, hysteresis, acceptable range, and PID gains.
ON-OFF controller with hysteresis	Drives the heater state when the application operates in Part 1 or hybrid mode.
PID controller	Computes the cooling command as a continuous duty-cycle percentage when the application operates in Part 2 or hybrid mode.
Heater resistor and LED	Represent the binary actuation path for heating.
L293D and DC motor fan	Represent the PWM-driven cooling actuator.
LCD1602	Displays measured temperature, setpoint, control mode, heater state, and fan duty.
Serial STDIO and Plotter output	Provide runtime observability and control-response visualization.
Sequential scheduler	Executes acquisition, setpoint processing, control, actuation, and reporting with justified recurrences.

2.4 2.4 Structural architectural design

The application architecture is organized into four logical layers:

- **Application layer:** scheduler and shared application state.
- **Task layer:** acquisition, setpoint management, control, actuation, and reporting tasks.
- **Service layer:** thermistor conversion, ON-OFF controller, PID controller, and serial command logic.
- **Driver layer:** thermistor input, potentiometer input, heater output, fan motor driver, LCD, and serial STDIO.

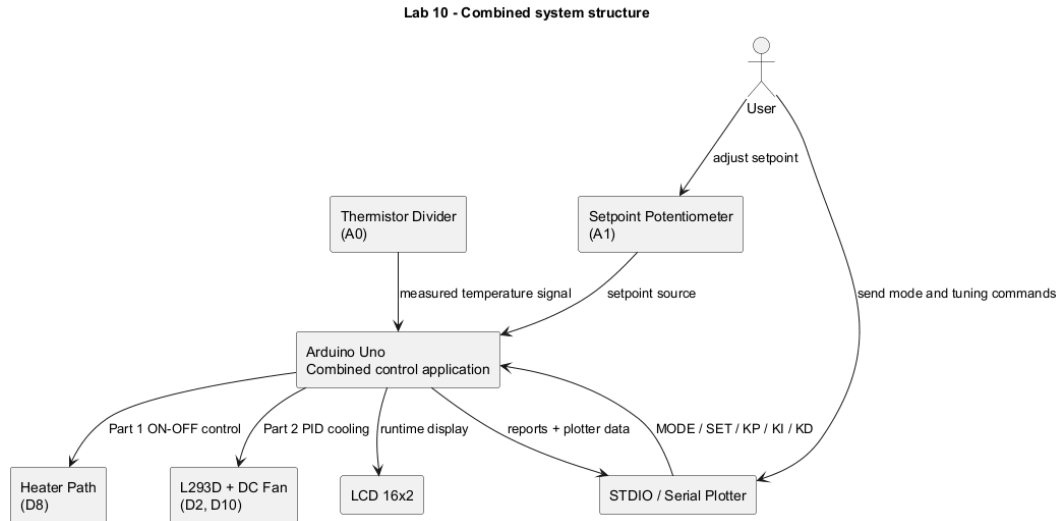


Figure 1: System structure for the combined Lab 10 control application

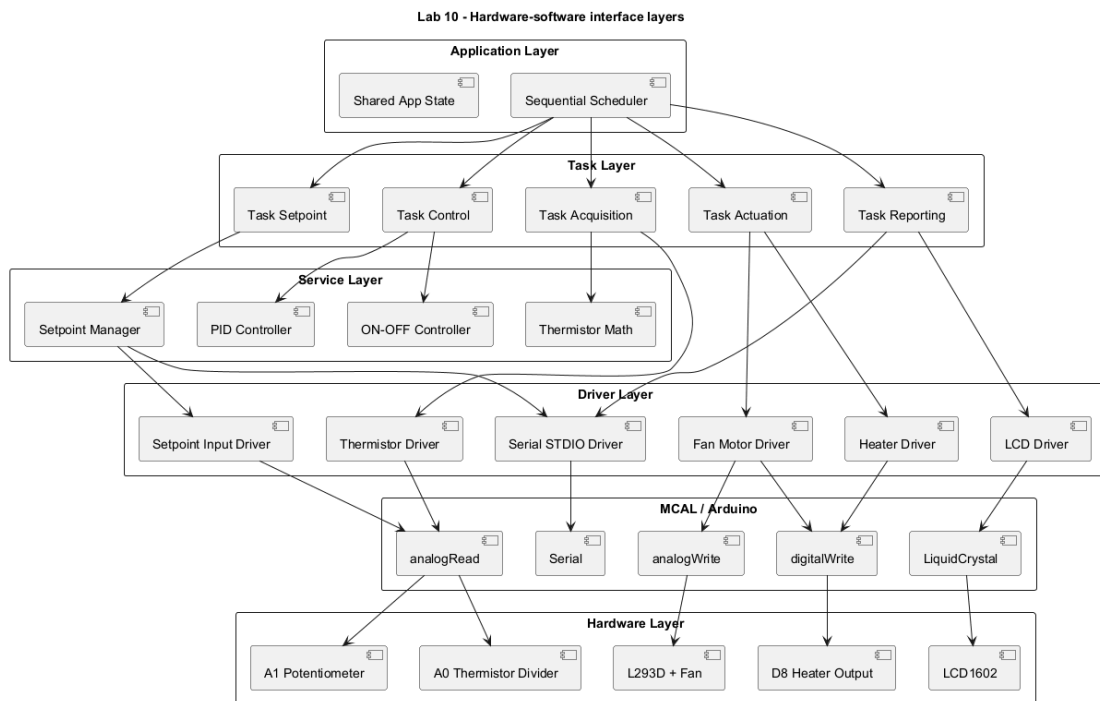


Figure 2: Hardware-software interface and layered decomposition

These diagrams satisfy the documentation requirement to present both the complete system and the HW/SW interface using a standardized format.

2.5 Behavioral modeling

The application uses one acquisition path and one reporting path, while the control path depends on the selected mode. In P1, the heater is controlled through ON-OFF hysteresis. In P2, the fan duty is computed through PID based on positive thermal error above the

setpoint. In HYBRID, the heater is allowed below the lower hysteresis threshold and the fan is activated only when the temperature exceeds the upper threshold.

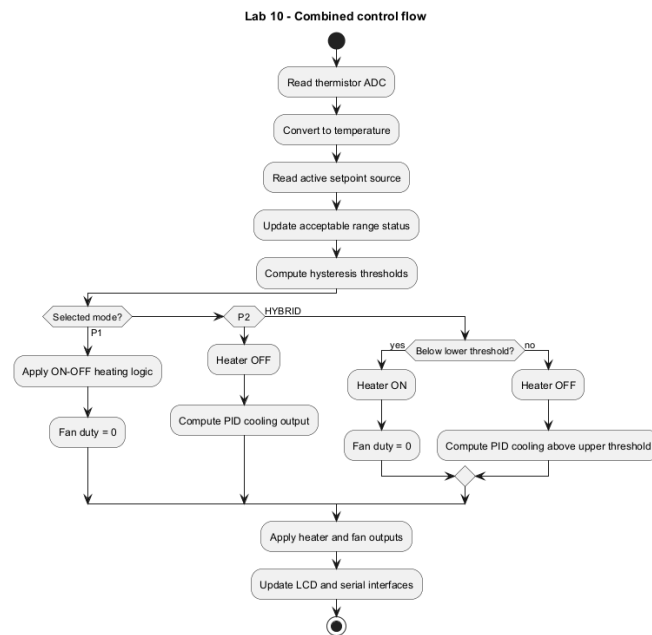


Figure 3: Combined control flow for the mode-selectable Lab 10 application

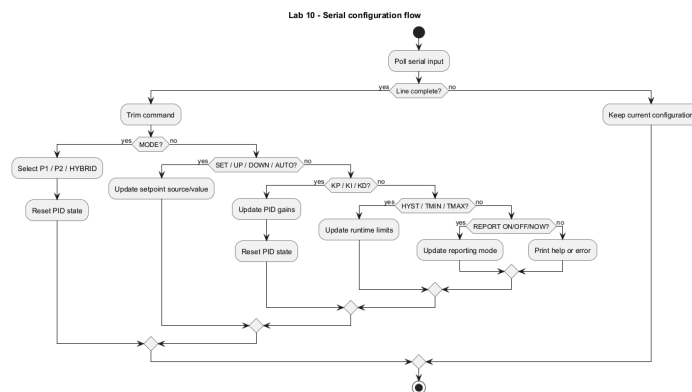


Figure 4: Serial configuration and runtime control flow

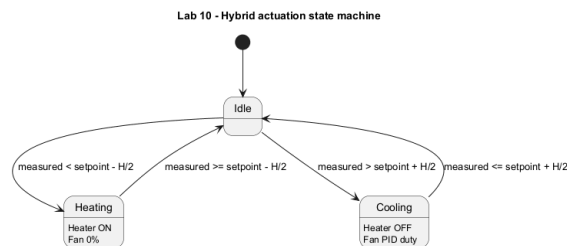


Figure 5: Actuation state machine in the hybrid heating/cooling mode

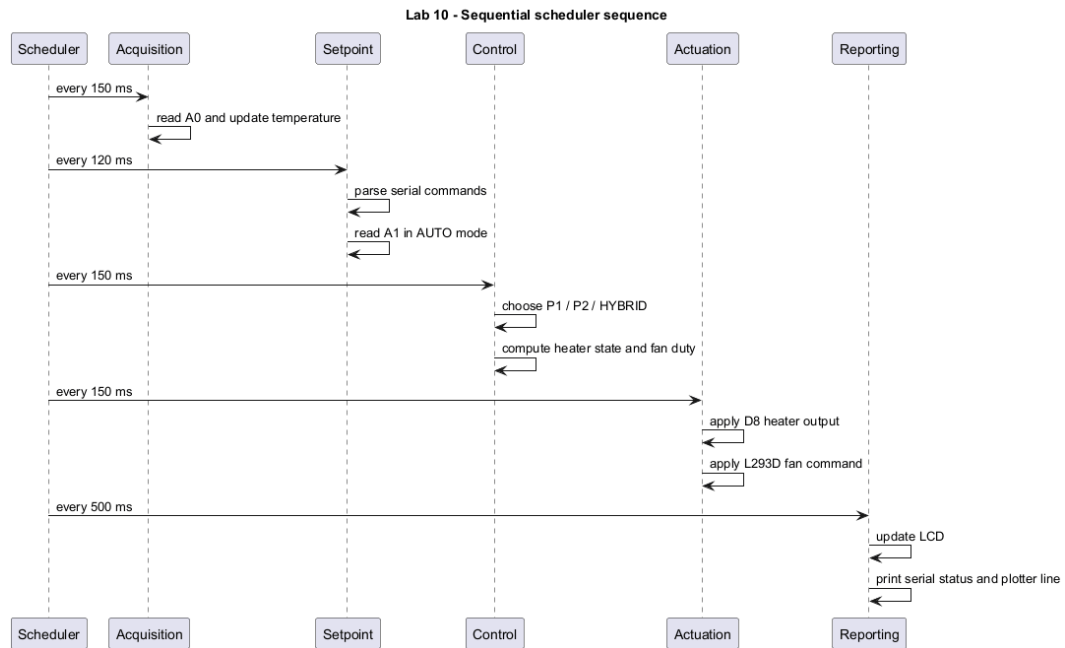


Figure 6: Sequential task execution schedule used in Lab 10

The selected behavioral diagrams cover flow, state logic, and runtime sequence, which are the main diagram categories requested by the guideline.

2.6 Test scenarios and validation criteria

ID	Scenario	Req.	Pass criterion
T1	Modify the simulated thermistor input and observe the measured temperature.	R1	measured value changes in reports
T2	Rotate the setpoint potentiometer in automatic mode.	R2	setpoint follows hardware input
T3	Switch among <code>MODE P1</code> , <code>MODE P2</code> , and <code>MODE HYBRID</code> .	R3, R4, R5	mode changes are accepted and reflected in reports
T4	Change PID gains through <code>KP</code> , <code>KI</code> , and <code>KD</code> .	R6	gains update and appear in runtime output
T5	Change <code>TMIN</code> and <code>TMAX</code> .	R7	acceptable range updates correctly
T6	In <code>P1</code> , set temperature below the lower threshold.	R3	heater turns ON
T7	In <code>P2</code> , set temperature above the setpoint.	R4	fan duty increases above zero
T8	In <code>HYBRID</code> , move temperature below and above the regulation band.	R5	heater and fan act in the correct regions
T9	Observe LCD, terminal, and Serial Plotter output.	R8, R9	values are readable and plotter fields are generated
T10	Inspect the source tree and modules.	R10	tasks, services, and drivers are separated
T11	Review the periodic scheduler implementation.	R11	justified sequential recurrences are explicit
T12	Run <code>pio run</code> .	R12	build succeeds
T13	Review the final code and behavior.	R13	implementation stays inside the control-focused scope

Table 3: Validation scenarios for Lab 10

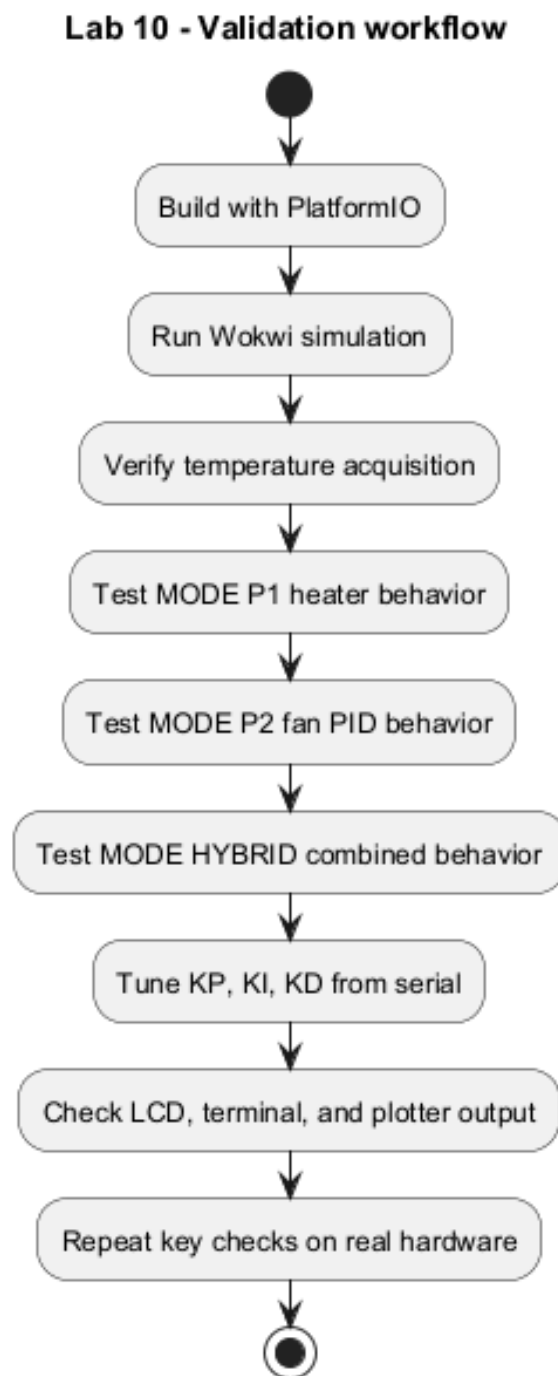


Figure 7: Validation workflow used for Lab 10

Design phase conclusions

The design stage produced a coherent architecture for integrating two control strategies into one application, while preserving traceability between requirements, diagrams, implementation decisions, and validation scenarios.

3. Hardware and software implementation

3.1 Electrical configuration

The final hardware combines one sensing chain, one reference input, and two actuation paths:

- thermistor divider on A0;
- setpoint potentiometer on A1;
- heating resistor and LED on D8;
- DC motor fan through L293D, commanded by D2 and D10;
- LCD1602 on D7, D6, D5, D4, D3, and D13;
- serial USB interface at 115200 baud.

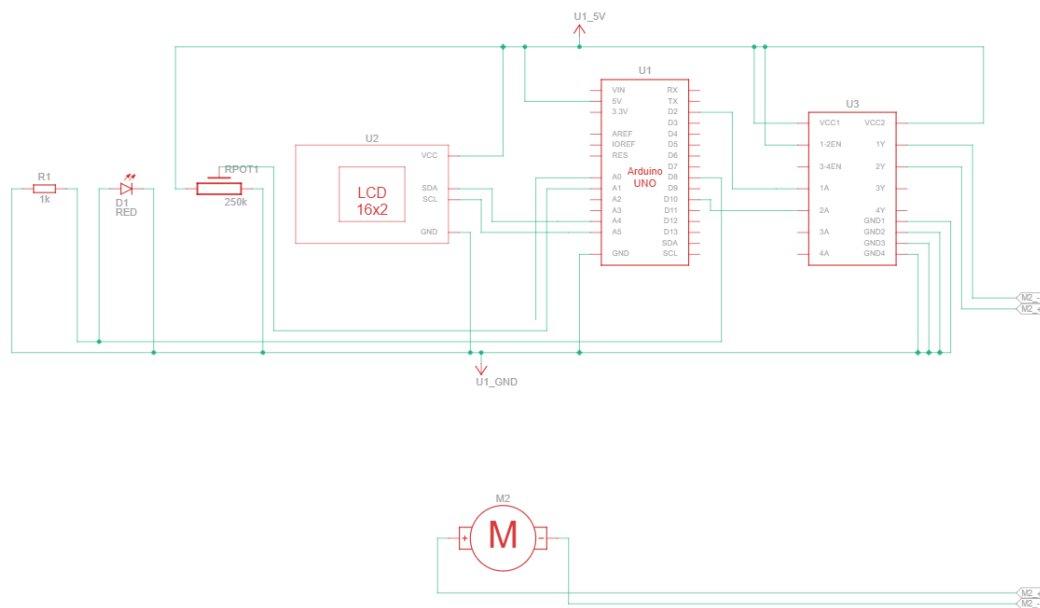


Figure 8: Electrical schematic of the Lab 10 combined heating/cooling setup

3.2 Source code organization

The software is organized modularly under Lab10/Code:

- Lab10/Code/src/main.cpp: program entry point.
- Lab10/Code/src/app/app_controller.cpp: initialization and sequential scheduling.
- Lab10/Code/src/app/app_state.cpp: shared runtime state.

- Lab10/Code/src/tasks/task_acquisition.cpp: signal acquisition and temperature update.
- Lab10/Code/src/tasks/task_setpoint.cpp: setpoint and command update.
- Lab10/Code/src/tasks/task_control.cpp: mode-dependent control logic.
- Lab10/Code/src/tasks/task_actuation.cpp: heater and fan output update.
- Lab10/Code/src/tasks/task_reporting.cpp: LCD, terminal, and plotter reporting.
- Lab10/Code/src/services/thermistor_math.cpp: divider-to-temperature conversion.
- Lab10/Code/src/services/on_off_controller.cpp: hysteresis-based control logic.
- Lab10/Code/src/services/pid_controller.cpp: cooling PID computation.
- Lab10/Code/src/services/setpoint_manager.cpp: serial interpreter and runtime configuration.
- Lab10/Code/src/drivers/*.cpp: low-level drivers for sensor input, LCD, serial, heater, and fan motor output.

3.3 Critical implementation sections

Sequential scheduler

```
1 ScheduledTask g_tasks[] = {  
2     {taskAcquisitionRun, kAcquisitionPeriodMs, OUL},  
3     {taskSetpointRun, kSetpointPeriodMs, OUL},  
4     {taskControlRun, kControlPeriodMs, OUL},  
5     {taskActuationRun, kActuationPeriodMs, OUL},  
6     {taskReportingRun, kReportingPeriodMs, OUL},  
7 };
```

Mode-dependent control branch

```
1 switch (g_appState.control.mode) {  
2     case ControlMode::Part1OnOff:  
3         heaterState = onOffState;  
4         break;  
5  
6     case ControlMode::Part2Pid:  
7         fanDutyPercent = pidControllerComputeCoolingPercent(...);  
8         break;  
9  
10    case ControlMode::Hybrid:
```

```
11 heaterState = onOffState;
12 fanDutyPercent = pidControllerComputeCoolingPercent(...);
13 break;
14 }
```

PID cooling computation

```
1 *integral += errorC * dtSeconds;
2 const float derivative = (errorC - *previousError) / dtSeconds;
3 const float output = (kp * errorC) + (ki * (*integral)) + (kd * derivative);
4 return clampFloat(output, 0.0F, 100.0F);
```

L293D fan drive

```
1 digitalWrite(kMotorDriverIn1Pin, LOW);
2 analogWrite(kMotorDriverIn2Pin, map(g_fanDutyPercent, 0, 100, 0, 255));
```

Runtime commands

```
1 MODE P1 / MODE P2 / MODE HYBRID
2 KP <value> / KI <value> / KD <value>
3 SET <temp> / HYST <value> / TMIN <value> / TMAX <value>
4 REPORT ON / REPORT OFF / REPORT NOW
```

These excerpts highlight the core implementation differences introduced by Lab 10: mode selection, PID cooling, and the coexistence of binary and analog-like actuation paths.

3.4 Implementation reasoning

The final implementation keeps one firmware for both parts because the same sensing, interaction, and reporting infrastructure can support both ON-OFF and PID control. This approach reduces duplication and makes the behavioral differences easier to demonstrate during testing.

The fan driver is based on L293D and PWM on D10. This allows the cooling effort to scale with the thermal error, which is essential for Part 2. The heater remains a binary output on D8, preserving the simplicity of Part 1 and making the hybrid mode meaningful.

Implementation phase conclusions

The implementation phase produced a modular and testable application that demonstrates both coarse and fine automatic control strategies with minimal hardware duplication and clear runtime observability.

4 4. Testing and validation

4.1 4.1 Validation methodology

Validation combines compilation and hardware observations:

1. build the project with `pio run`;
2. vary the measured temperature and observe the mode-dependent response;
3. switch control modes and update PID gains through serial commands;
4. observe heater behavior in `P1`, fan behavior in `P2`, and combined behavior in `HYBRID`;
5. inspect LCD output, serial status lines, and Serial Plotter data;
6. verify the real-hardware wiring of the thermistor divider, heater output, L293D, fan, and LCD.

4.2 Validation results

ID	Check	Status	Observation
T1	Measured-value acquisition	Pass	analog feedback is converted into temperature reports
T2	Setpoint control	Pass	potentiometer and serial setpoint work correctly
T3	Mode switching	Pass	P1, P2, and HYBRID are accepted at runtime
T4	PID gain update	Pass	KP/KI/KD modify the runtime configuration
T5	Acceptable range update	Pass	TMIN/TMAX thresholds update correctly
T6	Part 1 heating response	Pass	heater turns ON below the lower threshold
T7	Part 2 cooling response	Pass	fan duty increases above setpoint in PID mode
T8	Hybrid control response	Pass	heater and fan act in their respective temperature regions
T9	LCD and serial reporting	Pass	runtime values remain visible and structured
T10	Serial Plotter output	Pass	SetPoint, Value, and signed Output are available
T11	Modular structure review	Pass	tasks, services, and drivers are clearly separated
T12	Build verification	Pass	PlatformIO build succeeds
T13	Scope review	Pass	the final application remains focused on automatic control

Table 4: Validation summary for Lab 10

4.3 Proof of operation

The course documentation expects proof of operation in the form of terminal screenshots and hardware photos. The collected proof of operation includes:

- `terminal.png`: serial monitor output showing mode, temperature, heater, and fan duty;
- `hardware.jpg`: real hardware assembly photo;
- video link: optional demonstration of live mode switching and control behavior.

```

Commands:
SET <temp> - set serial setpoint in range 18.0..45.0
UP         - increase serial setpoint by 0.5 C
DOWN       - decrease serial setpoint by 0.5 C
AUTO       - use potentiometer as setpoint source
MODE P1    - Part 1 heater ON/OFF mode
MODE P2    - Part 2 PID fan mode
MODE HYBRID - heater ON/OFF + PID fan
HYST <value> - set hysteresis width in range 0.2..10.0 C
TMIN <value> - set acceptable minimum temperature
TMAX <value> - set acceptable maximum temperature
KP <value>   - set PID proportional gain
KI <value>   - set PID integral gain
KD <value>   - set PID derivative gain
REPORT ON   - enable periodic reporting
REPORT OFF  - disable periodic reporting
REPORT NOW  - print one report immediately
HELP        - print this help

Plotter fields: SetPoint Value Output
Mode=HYBRID Input=POT Source=pot SetPoint=20.24C Value=25.22C Hyst=1.40C HLow=19.54C HHigh=20.94C Tmin=15.00C Tmax=50.00C Range=OK Heater=OFF Fan=100%
Kp=35.00 Ki=0.80 Kd=12.00
SetPoint:20.24 Value:25.22 Output:-100
Mode=HYBRID Input=POT Source=pot SetPoint=20.24C Value=25.22C Hyst=1.40C HLow=19.54C HHigh=20.94C Tmin=15.00C Tmax=50.00C Range=OK Heater=OFF Fan=100%
Kp=35.00 Ki=0.80 Kd=12.00
SetPoint:20.24 Value:25.22 Output:-100

```

Figure 9: Serial terminal output during combined Part 1 and Part 2 operation

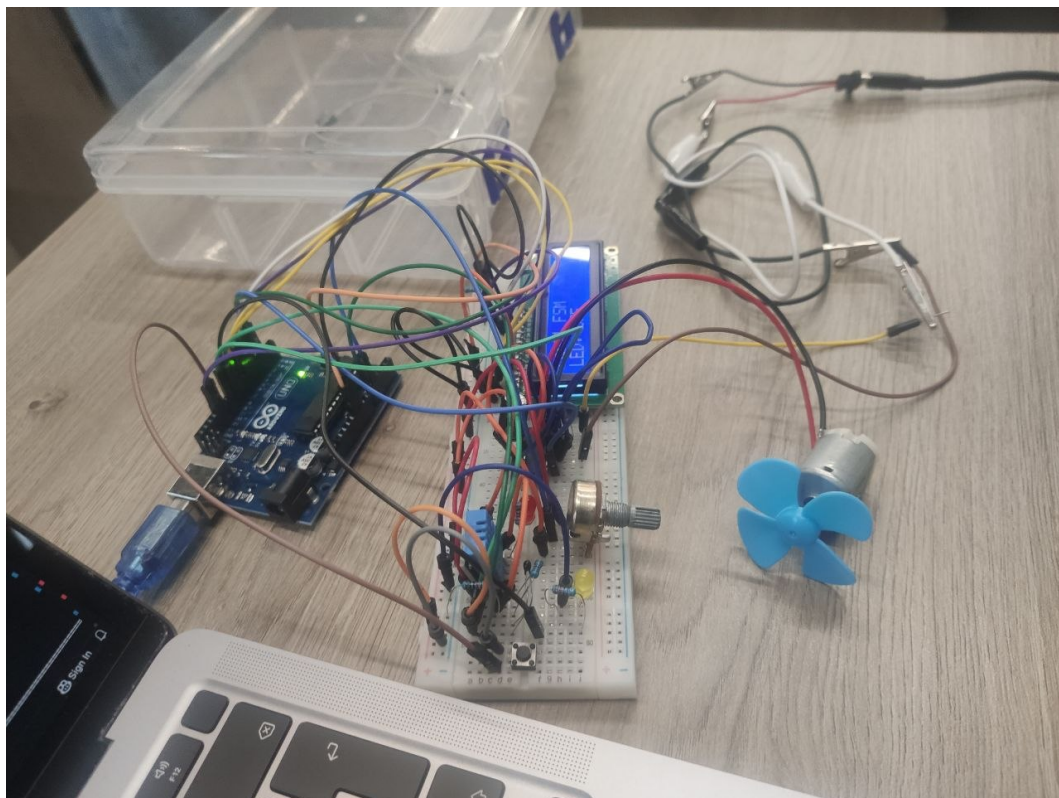


Figure 10: Real hardware assembly used for thermistor, heater, and fan control

4.4 Hardware observations

Real-hardware testing is essential for observing the coupling between the heater resistor, thermistor, and cooling fan. The hardware observations show that the heating resistor can influence the thermistor quickly if placed too close, while PID fan cooling provides a smoother response than simple binary switching above the setpoint.

Testing phase conclusions

Testing confirms that the final firmware successfully demonstrates both parts of the laboratory in a unified structure. The main limitations are related to the physical thermal setup rather than the software architecture itself, which remains configurable, modular, and observable.

General conclusions

The final Lab 10 project implements Part 2 PID cooling control on Arduino Uno while also preserving Part 1 ON-OFF heating logic and exposing a hybrid mode. This makes the application suitable both for laboratory compliance and for comparing binary and continuous control strategies using the same sensor, display, and serial interface.

The report follows the course documentation structure, includes technical requirements, architectural and behavioral diagrams, an exported electrical schematic, validation criteria, implementation details, and proof-of-operation evidence. The resulting system demonstrates that PID control is better suited for smooth cooling behavior, while ON-OFF control remains practical for simple heating actuation.

Possible improvements include adding persistent storage for PID gains and setpoint, introducing filtered thermistor acquisition, using a more realistic thermal plant, and extending the system with logging or adaptive tuning methods.

AI tool usage note

During the preparation of this report, the author used OpenAI GPT-based tooling for content generation and restructuring. The resulting information was reviewed, validated, and adjusted according to the project requirements before final inclusion.

Bibliography

1. Embedded Systems Practical Guide, Technical University of Moldova, 2026.
2. Laboratory specification for automatic control in embedded systems, Part 1 and Part 2.
3. PlatformIO Documentation, <https://docs.platformio.org/>.
4. Arduino Language Reference, <https://www.arduino.cc/reference/en/>.
5. LiquidCrystal Library Documentation, <https://github.com/arduino-libraries/LiquidCrystal>.
6. PlantUML Language Reference, <https://plantuml.com/>.

7. PID control references and educational materials on proportional-integral-derivative regulation.

Appendix

A. Project artifacts

The implementation is stored in `Lab10/Code/`.

B. GitHub repository

Repository URL: `https://github.com/Tirppy/si-course-repo/tree/main/Lab10/Code`

C. Diagram sources

PlantUML source files are stored in `Lab10/Report/diagrams/`. Rendered PlantUML images are stored in `Lab10/Report/images/`.

D. Build and run commands

- Build: `pio run` from `Lab10/Code`
- Serial monitor: `pio device monitor -b 115200`